Static Members in C#

A static member belongs to the class itself, rather than to a specific object of the class.

This means you can access a static member using the class name, without creating an object.

Types of Static Members

Common static members in C# include:

Static variables

Static methods

Static properties

Static classes

Static constructors

```
using System;

class Student
{
    public string Name;

    // Static variable shared by all Student objects
    public static int StudentCount = 0;

    public Student(string name)
    {
        Name = name;
        StudentCount++;
    }

    // Static method
    public static void DisplayStudentCount()
    {
        Console.WriteLine("Total Students: " + StudentCount);
```

```
    }
}

class Program
{
    static void Main()
    {
        Student s1 = new Student("Rahul");
        Student s2 = new Student("Priya");
        Student s3 = new Student("Amit");

        // Accessing static method using class name
        Student.DisplayStudentCount();

        Console.ReadLine();
    }
}
```

Another Simple Example

```
class Calculator
{
    public static int Add(int a, int b)
    {
        return a + b;
    }
}

class Program
{
    static void Main()
    {
```

```
        int result = Calculator.Add(10, 20);

        Console.WriteLine("Sum = " + result);
    }
}
```

.....

Encapsulation in C#

Encapsulation is an important concept of Object-Oriented Programming (OOP). It means wrapping data and methods together inside a class and controlling direct access to the data.

In C#, encapsulation is commonly achieved using private fields and public properties or methods.

```
using System;

class BankAccount
{
    // Private field - cannot be accessed directly outside the class
    private double balance;

    // Public property
    public double Balance
    {
        get { return balance; }
    }

    // Method to deposit money
    public void Deposit(double amount)
    {
        if (amount > 0)
        {
```

```
            balance += amount;
            Console.WriteLine("Amount deposited successfully.");
        }
        else
        {
            Console.WriteLine("Invalid amount.");
        }
    }

    // Method to withdraw money
    public void Withdraw(double amount)
    {
        if (amount > 0 && amount <= balance)
        {
            balance -= amount;
            Console.WriteLine("Amount withdrawn successfully.");
        }
        else
        {
            Console.WriteLine("Insufficient balance or invalid amount.");
        }
    }
}

class Program
{
    static void Main()
    {
        BankAccount account = new BankAccount();

        account.Deposit(5000);
```

```
        account.Withdraw(1500);

        Console.WriteLine("Current Balance: " + account.Balance);

        Console.ReadLine();
    }
}
```

.....

Polymorphism in C#

Polymorphism is an important concept of Object-Oriented Programming (OOP). The word means “many forms.”

In C#, polymorphism allows the same method or interface to perform different actions depending on the object or parameters.

There are two common types:

Compile-time polymorphism – Method overloading

Run-time polymorphism – Method overriding

1. Compile-Time Polymorphism: Method Overloading

The same method name can have different parameters.

```
using System;

class Calculator
{
    // Method with two parameters
    public int Add(int a, int b)
    {
        return a + b;
    }

    // Same method name with three parameters
```

```
    public int Add(int a, int b, int c)
    {
        return a + b + c;
    }
}

class Program
{
    static void Main()
    {
        Calculator calc = new Calculator();

        Console.WriteLine("Sum of 2 numbers: " + calc.Add(10, 20));
        Console.WriteLine("Sum of 3 numbers: " + calc.Add(10, 20, 30));

        Console.ReadLine();
    }
}
```

2. Run-Time Polymorphism: Method Overriding

A child class can provide its own implementation of a method defined in the parent class.

```
using System;

class Animal
{
    public virtual void Sound()
    {
        Console.WriteLine("Animal makes a sound");
    }
}
```

```
class Dog : Animal
{
    public override void Sound()
    {
        Console.WriteLine("Dog barks");
    }
}

class Cat : Animal
{
    public override void Sound()
    {
        Console.WriteLine("Cat meows");
    }
}

class Program
{
    static void Main()
    {
        Animal a1 = new Dog();
        Animal a2 = new Cat();

        a1.Sound();
        a2.Sound();

        Console.ReadLine();
    }
}
```